

Programmation Web Frontend

Session 3 - Version 3.0

L2/X64/06/PWF/187628/26-27

1. Les bases du Web dynamique



1. Les bases du Web dynamique

- ✓ Le Web et JavaScript
- ✓ L'environnement, la console
- ✓ Les expressions
- ✓ Les variables et leurs types
- ✓ La conversion de types

JavaScript dans la page

Kits 1 et 2 : écrire, lire, cliquer

Les opérateurs, les conditions

Les tableaux

Les boucles

Chaînes et nombres

Les fonctions, la portée

Les objets

Le portfolio dans la page

Le dépôt du cours

<https://github.com/eloudsa/ifapme-x64-pwf>

Les slides

`slides/`

Le PDF de chaque session

Pour relire à votre rythme

Les labos

`session-3/labo-1/`

La page de départ : `index.html`,
`style.css`, `main.js`

Disponibles avant la session

Les solutions

`session-3/labo-1/solutions/labo-1-1.js`

Une solution par niveau

Publiées après la session

Un seul endroit pour tout retrouver : slides, labos et solutions

Les commandes git

- **git clone** : une seule fois, crée le dossier **ifapme-x64-pwf**
- **git pull** : à chaque session, récupère les nouveautés
- Après la session, un nouveau **git pull** apporte les solutions
- Copiez le dossier du labo hors du clone : vous travaillez sur cette copie

```
# Une seule fois : copier le dépôt sur votre ordinateur  
git clone https://github.com/eloudsa/ifapme-x64-pwf.git
```

```
# À chaque session : récupérer les nouveautés  
cd ifapme-x64-pwf  
git pull
```

```
# Un problème lors du git pull ? Essayez ceci  
git fetch && git reset --hard origin/main
```

Ce qui change aujourd'hui

Sessions 1 et 2

```
console.log(total)
```

Le résultat s'affiche dans la **console**

Node.js et Code Runner, dans VS Code

À partir d'aujourd'hui

```
...textContent = total
```

Le résultat s'affiche dans la **page**

Live Server et la console du navigateur

Au bloc 2

On détaille les instructions de la page

Sessions 10 à 15

Le langage ne change pas : seule la destination du résultat change

De la console à la page

Session 2 : dans la console

```
const firstName = "Sara"  
const projects = 12  
  
console.log(`${firstName} : ${projects} projets`)
```

Aujourd'hui : dans la page

```
const firstName = "Sara"  
const projects = 12  
  
document.querySelector("#summary").textContent =  
  `${firstName} : ${projects} projets`
```

- Les données et les calculs ne changent pas
- Seule la dernière ligne change : **où** afficher le résultat
- La console reste là pour vérifier et déboguer

Le kit de survie

Une recette

```
document.querySelector("#titre")  
  .textContent = "Bonjour"
```

Quelques instructions pour agir sur la page

Tout de suite

Vous l'utilisez dès aujourd'hui, sans tout comprendre

Chaque session ajoute une ou deux instructions

Expliqué plus tard

Chaque pièce est décortiquée au bloc 2

La session est annoncée à chaque fois

Rappel : éléments, id et classes

index.html : les éléments

```
<h1 id="title">Mon portfolio</h1>

<p class="intro">Trois projets récents</p>

<button id="like" class="btn btn-primary">
  J'aime
</button>
```

style.css : les sélecteurs

```
#title {
  color: #4a72b8;
}

.intro {
  color: #6b7280;
}
```

- Un **élément** : balise ouvrante, contenu, balise fermante
- Les **attributs** précisent l'élément : **id**, **class**, **href**, **src**...
- CSS cible un **id** avec **#**, une classe avec **.** : JavaScript fera pareil

id ou class ?

	id	class
Combien par page ?	Un seul élément porte cet id	Autant d'éléments que l'on veut
Par élément	Un seul id	Plusieurs classes : class="btn btn-primary"
En CSS	#title	.intro
Rôle principal	Lire ou modifier la valeur d'un élément	Appliquer un style à plusieurs éléments
En JavaScript	Attraper cet élément (dès aujourd'hui)	Changer l'apparence (session 4)

id : un élément unique, pour lire ou modifier sa valeur. Classe : plusieurs éléments, pour un style

Relier des fichiers à la page

```
<head>
  <meta charset="UTF-8">
  <title>Mon portfolio</title>

  <!-- CSS depuis un CDN (Bootstrap) -->
  <link rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">

  <!-- Votre CSS, dans le dossier du projet -->
  <link rel="stylesheet" href="style.css">

  <!-- Votre JavaScript -->
  <script src="main.js" defer></script>
</head>
```

CSS : `<link href="...">`. JavaScript : `<script src="...">`. Un chemin local ou une adresse Web

Live Server : votre page en direct

Pourquoi Live Server ?

Double-clic sur le fichier

`file:///Users/sara/pwf/index.html`

Le navigateur lit le fichier **sur le disque**

Aucun rechargement : il faut actualiser à la main après chaque modification

Avec Live Server

`http://127.0.0.1:5500/index.html`

Un petit **serveur Web** tourne sur votre ordinateur

La page se recharge seule à chaque enregistrement

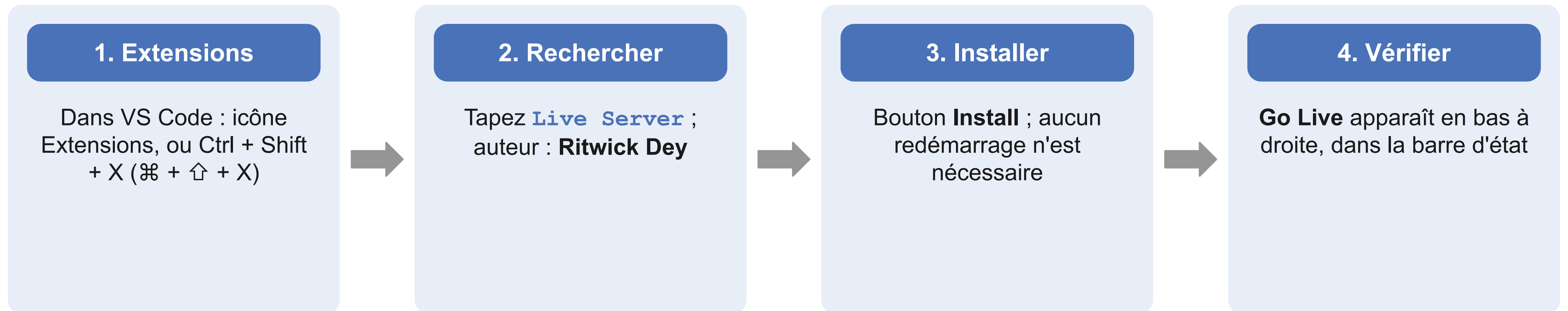
Le lien avec la session 1

Le navigateur envoie une **requête**, le serveur renvoie la **réponse**

Live Server joue le rôle du serveur, en local

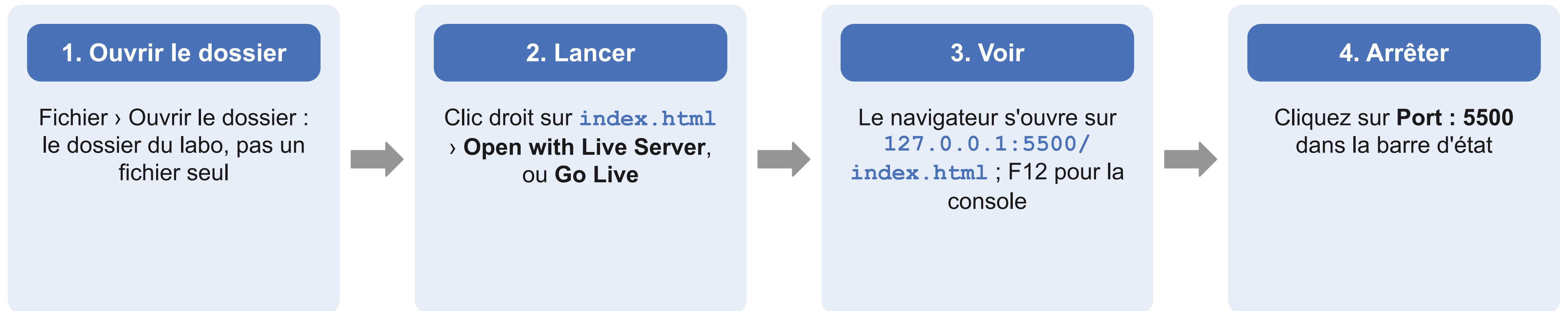
Votre page est servie comme un vrai site Web, et elle suit chacune de vos modifications

Installer Live Server



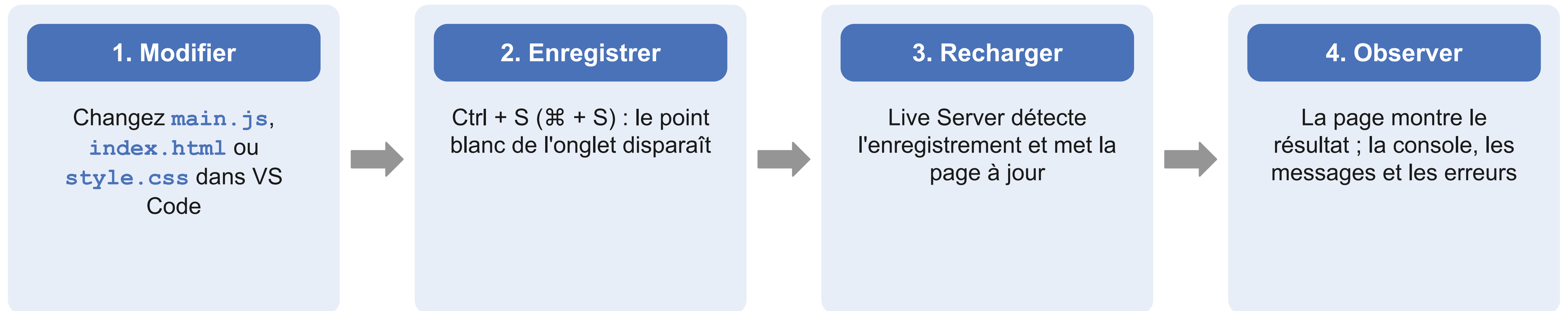
Installé en session 1 ? Vérifiez simplement la présence de « Go Live »

Lancer une page



Vérifiez l'adresse : `127.0.0.1`, c'est Live Server ; `file://`, c'est un double-clic

Modifier : la page suit



Chaque rechargement réexécute `main.js` depuis le début : les saisies dans les champs sont perdues

Live Server : problèmes fréquents

Ce que vous voyez	La cause probable	La solution
Pas de bouton Go Live	Extension absente, ou fichier ouvert seul	Installer ; ouvrir le dossier
L'adresse commence par file://	Page ouverte par un double-clic	Open with Live Server
La page ne change pas	Fichier non enregistré	Ctrl + S (⌘ + S)
Une autre page, ou une liste de fichiers	Serveur lancé depuis un autre dossier	Arrêter, relancer depuis le labo
Port 5501 au lieu de 5500	Un premier serveur tourne déjà	Arrêter l'ancien serveur

Premier contrôle : l'adresse dans le navigateur

Kit de survie 1 : le script et la page

Une page, un script

index.html : la structure

```
<head>
  <script src="main.js" defer></script>
</head>
<body>
  <h1 id="titre">Mon portfolio</h1>
  <p id="message">Bienvenue</p>
</body>
```

main.js : le comportement

```
console.log("Le script est branché")

document.querySelector("#titre").textContent = "Bonjour"
```

- **src** : le chemin du fichier JavaScript, comme **href** pour le CSS
- **defer** : attendre que toute la page soit lue avant d'exécuter le script
- Les **id** du HTML sont les poignées que le JavaScript attrape

Écrire dans la page

La première instruction qui change ce que voit l'utilisateur

```
document.querySelector("#titre").textContent = "Bonjour IFAPME"  
document.querySelector("#message").textContent = "Ce texte vient de main.js"
```

Bonjour IFAPME

Ce texte vient de main.js

APERÇU

Le texte écrit dans le HTML est remplacé au chargement de la page

Lire la recette

document

`document`

La page ouverte dans le navigateur

Le point de départ

querySelector

`.querySelector("#titre")`

Trouve l'élément dont l'`id` est `titre`

Le `#` est celui du CSS

textContent

`.textContent = "Bonjour"`

Remplace le texte de cet élément

Le `=` range la valeur de droite

Même sélecteur qu'en CSS : `#titre` désigne l'élément `id="titre"`

Kit de survie 1

Instruction	Ce qu'elle fait	En détail
<code><script src="main.js" defer></code>	Brancher le fichier JavaScript sur la page	S10
<code>document.querySelector("#id")</code>	Trouver un élément de la page	S10
<code>.textContent = "..."</code>	Remplacer le texte de cet élément	S10

Les concepts seront expliqués en détail plus tard

Rien ne se passe ?

Ce que vous voyez	La cause probable	La solution
La page ne change pas	Fichier non enregistré	Enregistrer (Ctrl + S, ⌘ + S)
Erreur 404 sur main.js	Chemin de src incorrect	Vérifier le nom du fichier
Cannot set properties of null	Élément introuvable	Vérifier le # et l'orthographe de l'id
SyntaxError	Guillemet ou parenthèse manquant	Lire la ligne indiquée dans la console

Le premier réflexe : ouvrir la console et lire les warnings et erreurs éventuels

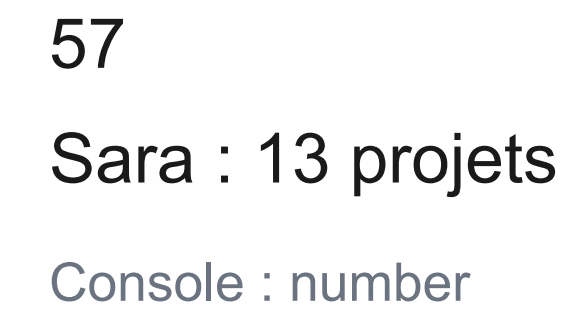
Vos acquis, dans la page

- Une expression est calculée, puis affichée
- Une variable garde la valeur ; **let** si elle change
- Un template literal assemble le texte à afficher
- La console vérifie ce que la page ne montre pas

```
// Une expression (session 1)
document.querySelector("#result").textContent = 19 * 3

// Des variables (session 2)
const firstName = "Sara"
let projects = 12
projects = projects + 1

// Un template literal (session 2)
document.querySelector("#summary").textContent =
  `${firstName} : ${projects} projets`
console.log(typeof projects)
```



57

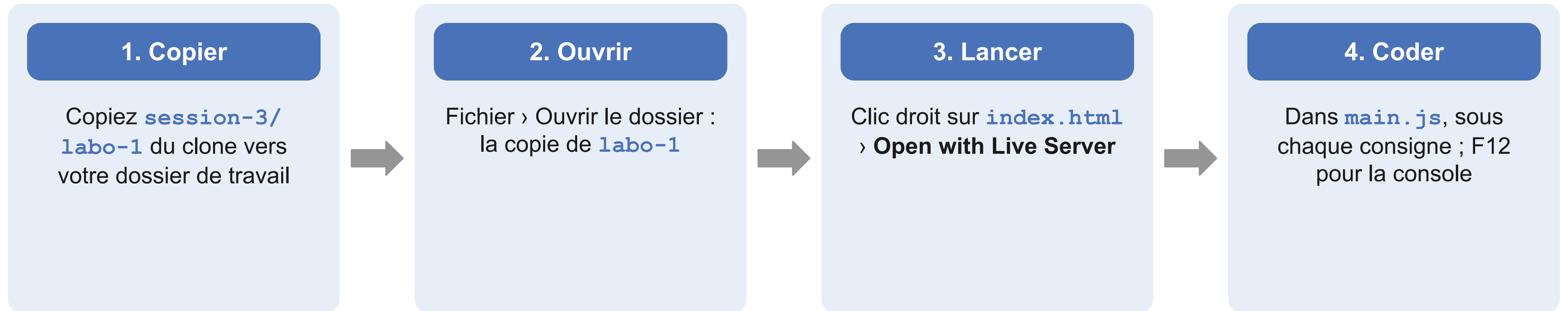
Sara : 13 projets

Console : number

APERÇU

Ouvrir un labo

Le dossier vient du dépôt du cours, récupéré avec git pull



Chaque labo arrive avec sa page : `index.html`, `style.css` et `main.js` (les consignes)

Labo 1 : la carte de visite

Niveau 1 · Simple

Des variables dans la page

- Live Server et la console
- Nom, métier, ville, e-mail en `const`
- Affichés dans la carte

Niveau 2 · Modéré

Le compteur et la phrase

- `let projects = 12`, puis + 1
- Une phrase en template literal
- Provoquer l'erreur d'une `const`

Niveau 3 · Avancé

L'inversion de palette

- Deux couleurs en `let`
- Les échanger sans retaper les valeurs
- Bonus : `typeof` dans la page

La page de départ

Le `<body>` de `index.html` : chaque id est une cible pour votre JavaScript

```
<main class="card business">
  <h1 id="name">Prénom Nom</h1>
  <p id="job" class="muted">Métier</p>
  <p id="city">Ville</p>
  <p id="email">adresse e-mail</p>
  <p><span id="projects">0</span> projets réalisés</p>
  <p id="summary" class="muted"></p>
  <div class="palette">
    <p>Couleur principale : <span id="primary">?</span></p>
    <p>Couleur secondaire : <span id="secondary">?</span></p>
  </div>
</main>
```

Prénom Nom

Métier

Ville

adresse e-mail

0 projets réalisés

Couleur principale : ?

Couleur secondaire : ?

APERÇU

Fournis : `index.html` et `style.css`. Vous écrivez uniquement dans `main.js`

Labo 1 · niveau 1

Niveau 1 · Simple

Des variables dans la page

- Ouvrez `index.html` avec Live Server, puis la console (F12)
- Déclarez en `const` votre nom complet, votre métier, votre ville et votre e-mail
- Affichez-les dans `#name`, `#job`, `#city` et `#email`
- Affichez aussi le nom et le métier dans la console

Hint

Les données en haut, l'affichage en dessous :

```
querySelector("#name").textContent = fullName
```

Labo 1 · niveau 2

Niveau 2 · Modéré

Le compteur et la phrase

- `let projects = 12` : affichez-le dans `#projects`
- Un nouveau projet est livré : ajoutez 1, puis réaffichez
- `#summary` : « Sara Lambert, Designer UX/UI à Liège » en template literal
- Réassignez votre métier (une `const`) : lisez l'erreur, puis commentez la ligne

Hint

Réassigner = réutiliser le nom, sans `let` : `projects = projects + 1`

Labo 1 · niveau 3

Niveau 3 · Avancé

L'inversion de palette

- `primaryColor` vaut "`#4A72B8`", `secondaryColor` vaut "`#F2994A`"
- Inversez les deux couleurs sans retaper les valeurs
- Affichez-les dans `#primary` et `#secondary`, et dans la console avant et après
- Bonus : affichez dans `#summary` le `typeof` de `projects` et de `primaryColor`

Hint

Si vous collez la couleur B sur A, où est passée A ? Il vous faut un troisième emplacement

Solution labo 1 · niveau 1

Niveau 1 · Simple

```
// Les données de la carte
const fullName = "Sara Lambert"
const job = "Designer UX/UI"
const city = "Liège"
const email = "sara@atelier.be"

// L'affichage dans la page
document.querySelector("#name").textContent = fullName
document.querySelector("#job").textContent = job
document.querySelector("#city").textContent = city
document.querySelector("#email").textContent = email

console.log(fullName, job)
```



Sara Lambert

Designer UX/UI

Liège

sara@atelier.be

Console : Sara Lambert Designer UX/UI

APERÇU

Solution labo 1 · niveau 2

Niveau 2 · Modéré

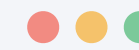
```
const fullName = "Sara Lambert"
const job = "Designer UX/UI"
const city = "Liège"

let projects = 12
document.querySelector("#projects").textContent = projects

// Un nouveau projet est livré
projects = projects + 1
document.querySelector("#projects").textContent = projects

document.querySelector("#summary").textContent =
  `${fullName}, ${job} à ${city}`

// job = "Développeuse"
// TypeError: Assignment to constant variable.
```



13 projets réalisés

Sara Lambert, Designer UX/UI à Liège

APERÇU

Solution labo 1 · niveau 3

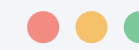
Niveau 3 · Avancé

```
let primaryColor = "#4A72B8"
let secondaryColor = "#F2994A"
console.log("Avant :", primaryColor, secondaryColor)

const temp = primaryColor
primaryColor = secondaryColor
secondaryColor = temp

document.querySelector("#primary").textContent = primaryColor
document.querySelector("#secondary").textContent = secondaryColor
console.log("Après :", primaryColor, secondaryColor)

// Bonus
const projects = 13
document.querySelector("#summary").textContent =
  `${typeof projects} / ${typeof primaryColor}`
```



Couleur principale : #F2994A

Couleur secondaire : #4A72B8

number / string

Console :

Avant : #4A72B8 #F2994A

Après : #F2994A #4A72B8

APERÇU

Kit de survie 2 : lire un champ, réagir au clic

Kit de survie 2

Instruction	Ce qu'elle fait	En détail
<code>const titre = document.querySelector(...)</code>	Garder l'élément dans une variable	S10
<code>champ.value</code>	Lire ce que l'utilisateur a saisi (une chaîne)	S13
<code>bouton.addEventListener("click", () => { ... })</code>	Exécuter un bloc à chaque clic	S11

Garder un élément dans une const

- Une variable peut contenir un **élément** de la page
- Cherché une fois, utilisé autant de fois que nécessaire
- **const** : c'est toujours le même élément, seul son texte change

```
// Kit 1 : chercher l'élément à chaque fois
document.querySelector("#projects").textContent = 12
document.querySelector("#projects").textContent = 13

// Kit 2 : le chercher une fois, le garder
const projectsText = document.querySelector("#projects")
projectsText.textContent = 12
projectsText.textContent = 13
```

13 projets réalisés

APERÇU

Réagir à un clic

- Le bloc entre accolades s'exécute à **chaque clic**
- Hors du bloc : une seule fois, au chargement
- Le mot "**click**" : en minuscules, entre guillemets
- Recette : la syntaxe `() => { ... }` sera expliquée en session 8

```
const likeButton = document.querySelector("#like")
const message = document.querySelector("#message")

likeButton.addEventListener("click", () => {
  message.textContent = "Merci pour votre like"
  console.log("Clic reçu")
})
```



J'aime

Merci pour votre like

Console : Clic reçu

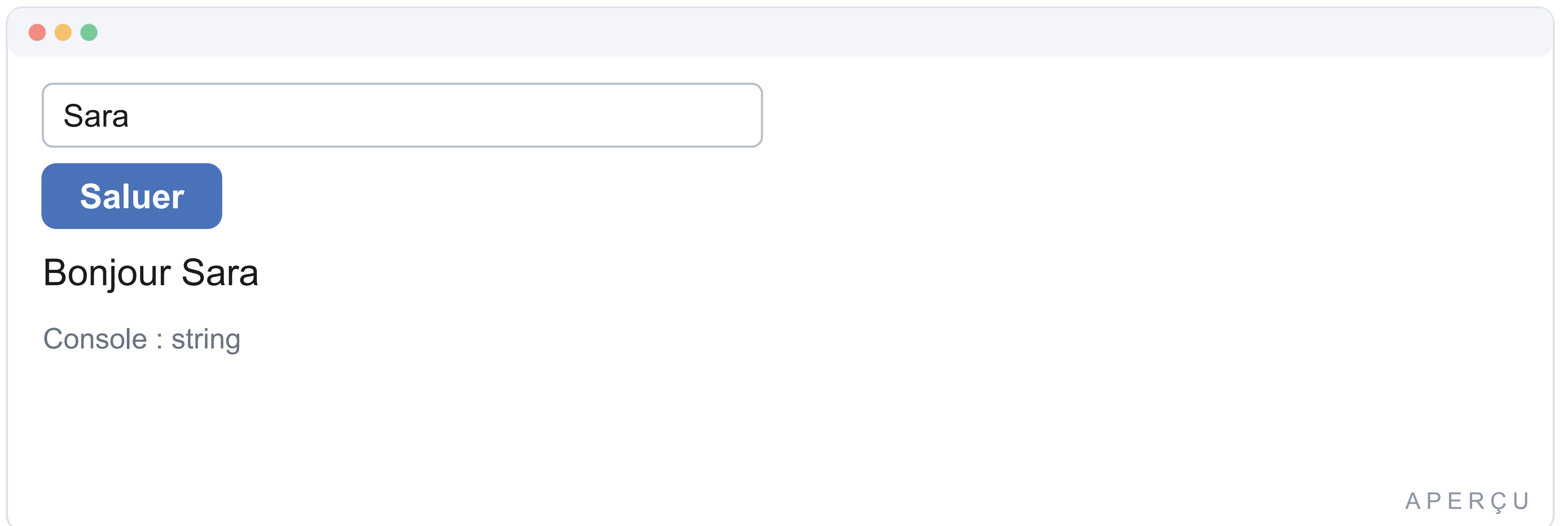
APERÇU

Lire un champ : value

- **champ.value** : le texte saisi, au moment où on le lit
- À lire **dans** le bloc du clic, pas au chargement
- Toujours une chaîne, même dans un champ **type="number"**

```
const nameInput = document.querySelector("#first-name")
const helloButton = document.querySelector("#hello")
const message = document.querySelector("#message")

helloButton.addEventListener("click", () => {
  const firstName = nameInput.value
  message.textContent = `Bonjour ${firstName}`
  console.log(typeof firstName)
})
```



Sara

Saluer

Bonjour Sara

Console : string

APERÇU

Le devis qui glisse

```
const designInput = document.querySelector("#design-days")
const devInput = document.querySelector("#dev-days")
const computeButton = document.querySelector("#compute")
const daysText = document.querySelector("#days-total")

computeButton.addEventListener("click", () => {
  daysText.textContent = designInput.value + devInput.value
})
```

Calculateur de devis

Jours de design

Jours de développement

Calculer

Total : 23 jours

APERÇU

value est une chaîne : "2" + "3" colle au lieu d'additionner

Convertir avant de calculer

La conversion de la session 2, appliquée aux saisies

Number()

`Number ("2")`

→ 2

Vers un nombre : avant tout calcul

Number("12px")

`Number ("12px")`

→ NaN

Not a Number : la saisie n'est pas un nombre

Number("")

`Number ("")`

→ 0

Un champ vide passe pour zéro, sans erreur

`Number (designInput.value) + Number (devInput.value) → 5`

Labo 2 : le calculateur de devis

Niveau 1 · Simple

Le total des jours

- Lire deux champs au clic
- Constater « 23 »
- Corriger avec `Number()`

Niveau 2 · Modéré

Le prix du devis

- Tarif journalier : 450 €
- Prix HTVA, puis TVAC
- Une phrase en template literal

Niveau 3 · Avancé

Le devis complet

- Le nom du client
- Une ligne de debug avec `typeof`
- Et si un champ est vide ?

La page de départ

Trois champs à lire, un bouton à surveiller, trois zones où écrire

```
<div class="card form">
  <label>Client
    <input id="client" type="text">
  </label>
  <label>Jours de design
    <input id="design-days" type="number" value="2">
  </label>
  <label>Jours de développement
    <input id="dev-days" type="number" value="3">
  </label>
  <button id="compute">Calculer</button>
</div>

<p>Total : <strong id="days-total">?</strong> jours</p>
<p id="price"></p>
<p id="debug" class="muted"></p>
```

Calculateur de devis

Client

Jours de design

Jours de développement

Calculer

Total : ? jours

APERÇU

Les champs de jours ont une valeur de départ (**value="2"**, **value="3"**) : un clic suffit pour tester

Labo 2 · niveau 1

Niveau 1 · Simple

Le total des jours

- Gardez dans des `const` les deux champs, le bouton `#compute` et `#days-total`
- Au clic, additionnez les deux `value` et affichez le résultat
- Avec 2 et 3, que voyez-vous ? Affichez le `typeof` d'une `value` dans la console
- Corrigez avec `Number()` : 5 jours

Hint

Faites d'abord l'erreur exprès :
c'est la console qui vous montrera
pourquoi

Labo 2 · niveau 2

Niveau 2 · Modéré

Le prix du devis

- En haut : tarif journalier (450) et taux de TVA (0.21)
- Au clic : prix HTVA = total des jours x tarif
- Puis le prix TVAC
- `#price` : « 4500 € HTVA, soit 5445 € TVAC »

Hint

Une `const` par étape du calcul :
un résultat faux se repère tout de suite

Labo 2 · niveau 3

Niveau 3 · Avancé

Le devis complet

- Lisez aussi `#client` : faut-il le convertir ?
- `#price` : « Devis pour Friterie du Coin : 5 jours, 2722.5 € TVAC »
- `#debug` : « design : string → number »
- Videz un champ et cliquez : que devient le total ? Expliquez en commentaire

Hint

`typeof` sur la `value` brute, puis sur la valeur convertie

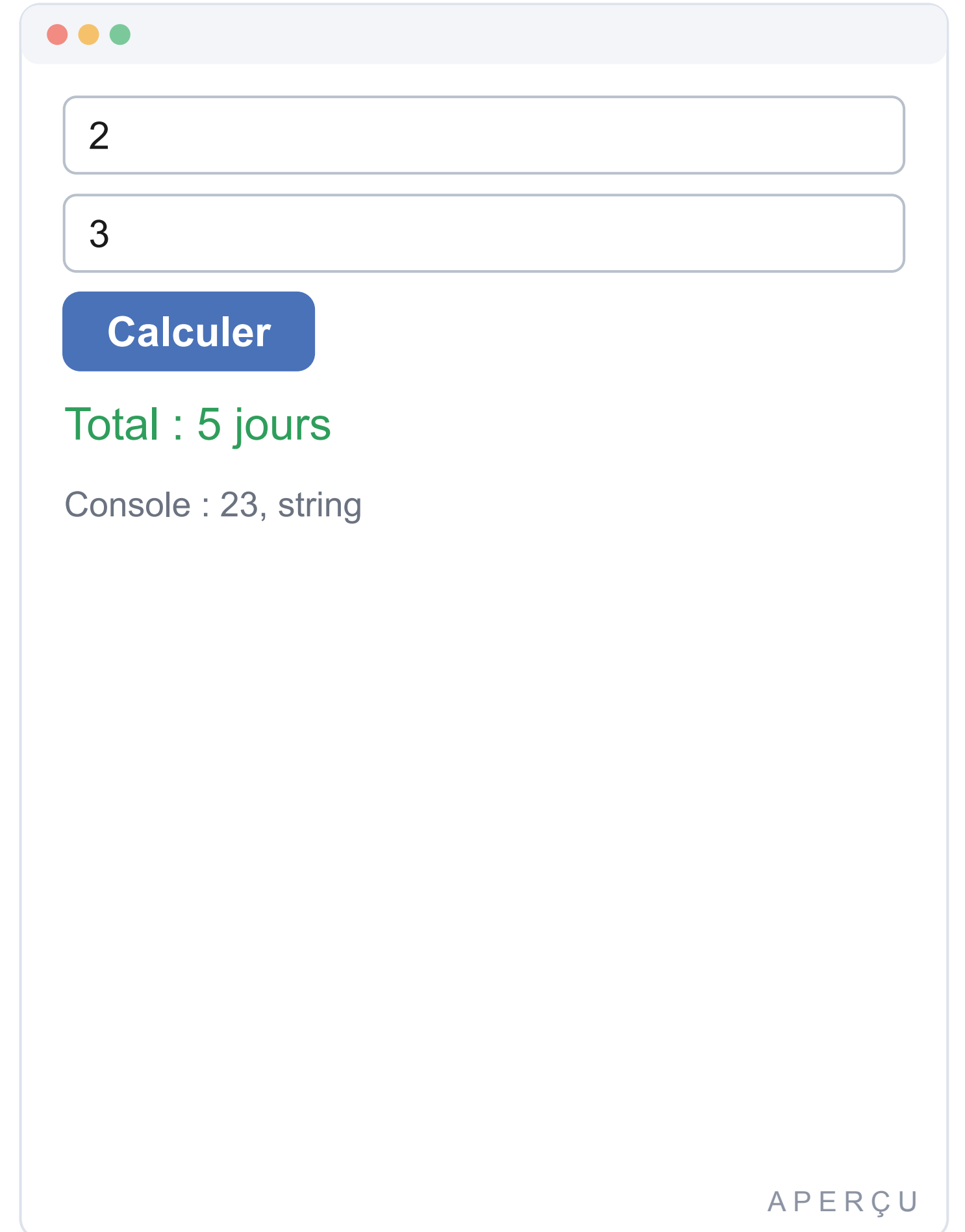
Solution labo 2 · niveau 1

Niveau 1 · Simple

```
const designInput = document.querySelector("#design-days")
const devInput = document.querySelector("#dev-days")
const computeButton = document.querySelector("#compute")
const daysText = document.querySelector("#days-total")

computeButton.addEventListener("click", () => {
  console.log(designInput.value + devInput.value)
  console.log(typeof designInput.value)

  const totalDays = Number(designInput.value) + Number(devInput.value)
  daysText.textContent = totalDays
})
```



2

3

Calculer

Total : 5 jours

Console : 23, string

APERÇU

Solution labo 2 · niveau 2

Niveau 2 · Modéré

```
const dailyRate = 450
const vatRate = 0.21

const designInput = document.querySelector("#design-days")
const devInput = document.querySelector("#dev-days")
const computeButton = document.querySelector("#compute")
const daysText = document.querySelector("#days-total")
const priceText = document.querySelector("#price")

computeButton.addEventListener("click", () => {
  const totalDays = Number(designInput.value) + Number(devInput.value)
  const price = totalDays * dailyRate
  const priceWithVat = price + price * vatRate

  daysText.textContent = totalDays
  priceText.textContent =
    `${price} € HTVA, soit ${priceWithVat} € TVAC`
})
```



A screenshot of a web application interface. It features two input fields at the top, one containing the number '4' and the other '6'. Below these is a blue button labeled 'Calculer'. Under the button, the text 'Total : 10 jours' is displayed, followed by '4500 € HTVA, soit 5445 € TVAC'. At the bottom right of the interface, the word 'APERÇU' is visible.

Solution labo 2 · niveau 3

Niveau 3 · Avancé

```
const dailyRate = 450
const vatRate = 0.21

const clientInput = document.querySelector("#client")
const designInput = document.querySelector("#design-days")
const devInput = document.querySelector("#dev-days")
const computeButton = document.querySelector("#compute")
const daysText = document.querySelector("#days-total")
const priceText = document.querySelector("#price")
const debugText = document.querySelector("#debug")

computeButton.addEventListener("click", () => {
  const client = clientInput.value
  const designDays = Number(designInput.value)
  const devDays = Number(devInput.value)
  const totalDays = designDays + devDays
  const price = totalDays * dailyRate * (1 + vatRate)

  daysText.textContent = totalDays
  priceText.textContent =
    `Devis pour ${client} : ${totalDays} jours, ${price} € TVAC`
  debugText.textContent =
    `design : ${typeof designInput.value} → ${typeof designDays}`
  console.log(client, totalDays, price)
})

// Champ vide : Number("") vaut 0, sans aucune erreur
```



Total : 3 jours

Devis pour Friterie du Coin : 3 jours, 1633.5 €
TVAC

design : string → number

Après avoir vidé le champ des jours de design

APERÇU

En résumé

La page

Le résultat s'affiche dans la page ; la console sert à vérifier

Live Server

Ouvrir `index.html`, enregistrer, la page se recharge

Kit 1

```
<script defer>,  
querySelector("#id"),  
textContent
```

Vos acquis

Expressions, variables, template literals : inchangés

Kit 2

Un élément en `const`, lire `.value`, réagir au clic

Conversion

`value` est une chaîne : `Number()` avant de calculer

Plus tard

Chaque recette est expliquée au bloc 2, à partir de la session 10

Prochaine session

Opérateurs et conditions ; kit 3 : classes et styles